GLOBAL WORK MARKETPLACE

AI + Humans

PROJECT PROPOSAL BY KIDENGE ELISHA

System Architecture Team

April 19, 2026

Contents

1 Vision **3**
1.1 Core Idea . . . 3
1.2 One-line definition . . . 3
1.3 Problem Being Solved . . . 3

2 System Overview **3**
2.1 High-Level Architecture . . . 3

3 Project Structure (Final Architecture) **4**

4 Core Engine (The Brain) **4**
4.1 Purpose . . . 4
4.2 Modules . . . 5
4.2.1 `router.py` . . . 5
4.2.2 `pricing_engine.py` . . . 5
4.2.3 `risk_engine.py` . . . 5
4.2.4 `optimizer.py` . . . 5
4.2.5 `decision_log.py` . . . 5

5 Dispatch System (Real-time Engine) **5**
5.1 Purpose . . . 5
5.2 Core Components . . . 5
5.2.1 `matcher.py` . . . 5
5.2.2 `realtime_queue.py` . . . 5
5.2.3 `priority_engine.py` . . . 6
5.2.4 `assignment.py` . . . 6

6 Task System **6**
6.1 Task Model Types . . . 6
6.2 Task Decomposition . . . 6

7 Agents System **6**
7.1 Definition . . . 6
7.2 Capabilities . . . 6
7.3 Health Check . . . 6

8 Execution Engine **6**
8.1 Purpose . . . 7
8.2 Subsystems . . . 7
8.2.1 `orchestrator.py` . . . 7
8.2.2 `sandbox/` . . . 7
8.2.3 `adapters/` . . . 7
8.2.4 `streaming/` . . . 7

9 Payments System (Very Critical) **7**
9.1 Architecture . . . 7
9.2 Escrow system . . . 7
9.3 Micropayments . . . 7
9.4 Settlement engine . . . 7

10 Verification System **7**
10.1 Purpose . . . 8
10.2 Components . . . 8
10.2.1 `consensus.py` . . . 8
10.2.2 `reputation_engine.py` . . . 8
10.2.3 `fraud_detection.py` . . . 8

11 Analytics System **8**

12 Webhook System **8**

13 Support System **8**

14 Key Innovation (Important) **8**

15 Core Value Proposition **9**

16 Risks / Challenges **9**

17 MVP Strategy (Important) **10**

18 Final Summary **10**

1 Vision

1.1 Core Idea

A real-time execution economy where:

- Humans and AI agents both provide services
- Work is broken into micro-tasks
- Tasks are routed instantly
- Execution is streamed in real-time
- Payment is automatically settled per execution unit

1.2 One-line definition

"An autonomous marketplace where work is dynamically assigned, executed, verified, and paid in real time by humans and AI agents."

1.3 Problem Being Solved

Current systems:

- **Upwork**: slow, manual freelancing
- **Fiverr**: static gig marketplace
- **Job boards**: delayed hiring cycles
- **AI tools**: isolated (no economic layer)

Gap: There is NO unified system where:

- AI agents can earn money directly
- Humans and AI collaborate dynamically
- Work is executed like "compute tasks"

2 System Overview

2.1 High-Level Architecture

The system is divided into 6 layers:

Layer	Purpose
L1 — Identity Layer	accounts, agents
L2 — Work Layer	tasks, dispatch
L3 — Intelligence Layer (CORE)	core_engine
L4 — Execution Layer	execution runtime, sandbox, AI + human adapters
L5 — Economic Layer	payments, escrow, ledger
L6 — Trust Layer	verification, fraud detection, reputation

3 Project Structure (Final Architecture)

```
global_work_marketplace/

 config/                  → Django system setup
 apps/                    → Business logic modules
 core_engine/            → AI decision brain
 execution/              → Task runtime engine
 dispatch/               → Real-time matching system
 payments/               → Financial system (ledger + escrow)
 tasks/                  → Work definition layer
 agents/                 → Humans + AI entities
 accounts/               → Users & authentication
 verification/           → Trust + reputation system
 analytics/              → System intelligence metrics
 webhooks/               → External integrations
 support/                → User support system
 common/                 → Shared utilities
```

4 Core Engine (The Brain)

Location: `core_engine/`

4.1 Purpose

Decision-making system that determines:

- Who executes a task (AI vs human)
- How much it costs
- Risk level

- Priority
- Execution strategy

4.2 Modules

4.2.1 `router.py`

Decides execution route: **Input:** Task, **Output:** AI | HUMAN | HYBRID Factors: complexity score, latency requirement, cost constraints, availability

4.2.2 `pricing_engine.py`

Dynamic pricing system:

$$\text{price} = \text{base_cost} \times \text{complexity} \times \text{urgency} \times \text{reputation_factor}$$

Adjusts in real-time based on supply/demand, agent scarcity, task urgency.

4.2.3 `risk_engine.py`

Evaluates fraud probability, agent reliability, task ambiguity. Output: $\text{risk_score} \in [0, 1]$.

4.2.4 `optimizer.py`

Global system optimization: reduces cost, reduces latency, balances load.

4.2.5 `decision_log.py`

Stores every decision: why AI was selected, why price changed, why task failed/succeeded.

5 Dispatch System (Real-time Engine)

Location: `dispatch/`

5.1 Purpose

Matches tasks to execution agents instantly.

5.2 Core Components

5.2.1 `matcher.py`

Matches based on embeddings similarity, skills overlap, availability, location (optional), past performance.

5.2.2 `realtime_queue.py`

Priority system with queues: urgent, normal, batch. Supports backpressure handling, dead-letter queue, retry logic.

5.2.3 `priority_engine.py`

Priority formula:

$$\text{priority} = \text{urgency} + \text{budget_weight} + \text{SLA_factor} + \text{user_tier}$$

5.2.4 `assignment.py`

Atomic task locking: prevents double assignment, supports timeout, reassigns failed tasks.

6 Task System

Location: `tasks/`

6.1 Task Model Types

Type	Description
Task	Full job unit
MicroTask	Broken sub-task
TaskTemplate	Reusable workflows

6.2 Task Decomposition

`decomposer.py` breaks "Build website" into: UI design, backend API, deployment. Uses AI decomposition engine.

7 Agents System

Location: `agents/`

7.1 Definition

An agent is either: Human worker, AI model (GPT, Claude, etc.), or Hybrid system.

7.2 Capabilities

Each agent has: skills vector, performance score, cost rate, response time, reliability score.

7.3 Health Check

`health_check.py`: heartbeat monitoring, offline detection, recovery routing.

8 Execution Engine

Location: `execution/`

8.1 Purpose

Runs tasks in real-time.

8.2 Subsystems

8.2.1 `orchestrator.py`

Controls lifecycle: `START` → `PROCESS` → `VERIFY` → `COMPLETE`

8.2.2 `sandbox/`

Secure execution environment: code isolation, docker containerization, resource limits, network restrictions.

8.2.3 `adapters/`

Connects external systems: OpenAI, Claude, Gemini, Human UI interface.

8.2.4 `streaming/`

Real-time output: WebSocket streaming, SSE (server-sent events).

9 Payments System (Very Critical)

Location: `payments/`

9.1 Architecture

- **Wallet system**: Each user/agent has wallet
- **Ledger system**: Double-entry accounting (debit/credit system)

9.2 Escrow system

Funds are: (1) locked, (2) held during execution, (3) released on verification.

9.3 Micropayments

Supports: per-second AI work billing, per-output payment, per-task completion fee.

9.4 Settlement engine

Instant Lightning payments (optional), batch settlement system, on-chain fallback.

10 Verification System

Location: `verification/`

10.1 Purpose

Ensures output quality.

10.2 Components

10.2.1 `consensus.py`

Multiple agents verify output.

10.2.2 `reputation_engine.py`

Score formula:

$$\text{reputation} = \text{success_rate} + \text{speed} + \text{quality} + \text{trust}$$

10.2.3 `fraud_detection.py`

Detects: fake outputs, collusion, spam agents.

11 Analytics System

Tracks: task success rate, AI vs human performance, revenue flow, system load.

12 Webhook System

Used for: external integrations, real-time notifications, event streaming.

13 Support System

Handles: disputes, refunds, user issues, task complaints.

14 Key Innovation (Important)

Key Innovation

This system is **not**:

- Freelancing platform
- AI API marketplace

It is: **A real-time economic execution layer for digital work**

15 Core Value Proposition

What makes this powerful:

1. **Work becomes atomic**: Tasks → micro-executable units
2. **AI becomes economic actor**: AI can earn money
3. **Real-time routing**: No waiting, no bidding delays
4. **Automatic pricing**: Dynamic supply-demand economy

16 Risks / Challenges

Risks / Challenges

- Extremely complex system
- High infrastructure cost
- Trust system hard
- Payment compliance (critical)
- AI reliability issues

17 MVP Strategy (Important)

MVP Strategy

Phase 1:

- tasks
- agents
- simple dispatch
- simple payments

Phase 2:

- core_engine
- AI routing
- micropayments

Phase 3:

- verification system
- fraud detection
- full real-time economy

18 Final Summary

"The economic operating system for AI + humans"

Not: marketplace, SaaS app, freelance tool.